RAG FROM SCRATCH

NLP Workshop



SETUP

1 · Get the workshop files

```
git clone https://github.com/NUS-SDS-Workshops-AY2627/Sem-1.git
cd Sem-1/week-11-nlp
python3 -m venv .venv # Windows: python
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

In VS Code: open **week-11-nlp** → **workshop.ipynb** → Select Kernel → Python Environments → **.venv**

Keep the folder as it is: the notebook needs rag_utils.py and the data folder.

2 · Get a Gemini API key

1. Go to aistudio.google.com/apikey, sign in with a personal Google account
2. Create API key and copy it. Don't set up billing.
3. Run the setup cells, paste the key when asked

Works if you see the module count and **Gemini says: ready**

Use your own key: a shared one runs out in minutes.



Team Members



Sara
Y1 DSAAI



Nabil
Y3 DSA



Dharshini
Y1 Masters AIS

READING TEXT

- How does a computer read text?
- Computers only understand numbers → we have to turn words into numbers
- Text is hard: it's messy and ambiguous
- The NLP pipeline:



- We take one NUSMods description as example:
  - **"This course introduces basic concepts and algorithms in machine learning and neural networks."**
  - ——CS3244

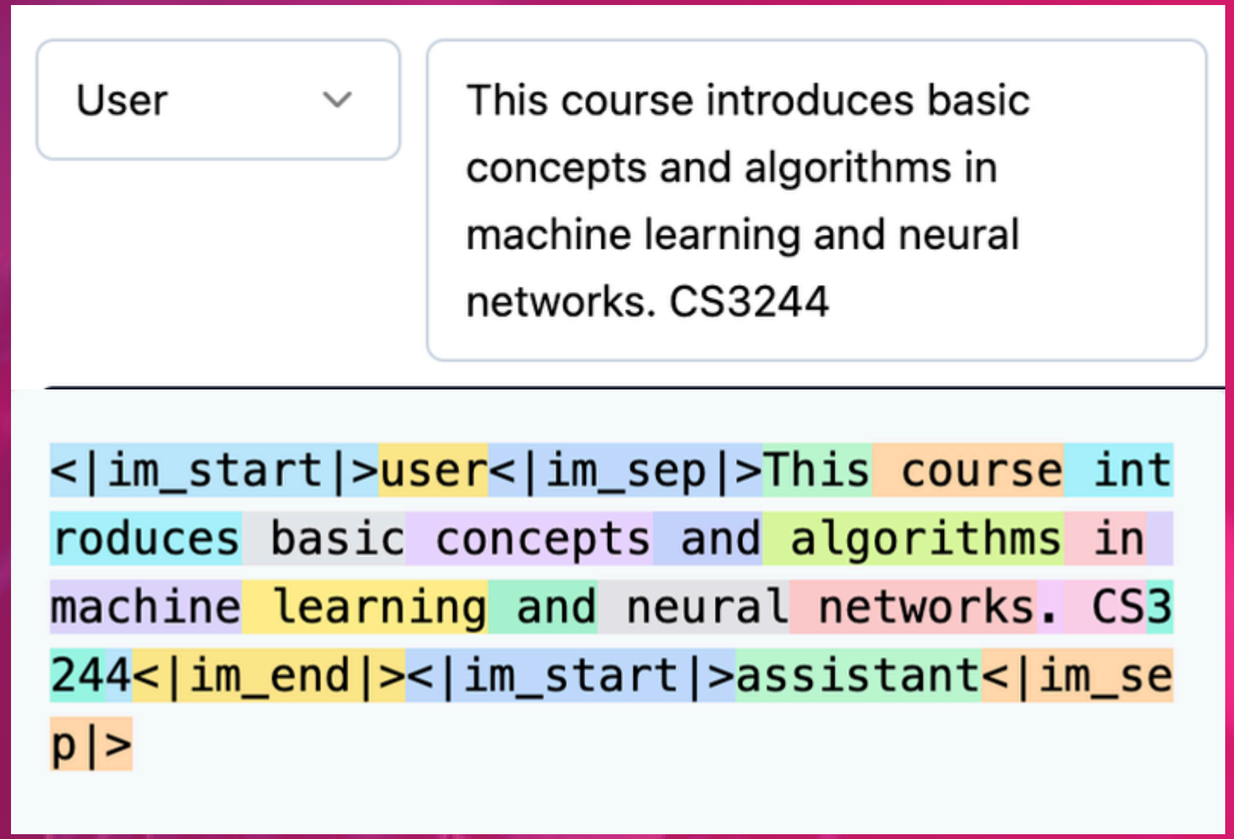
Cleaning, Tokenization, Normalization

PREPROCESSING

Step	What it does	Result
Raw text	original course description	This course introduces basic concepts and algorithms in machine learning and neural networks.
BEFORE tokenization · on the whole text		
Cleaning	remove punctuation & symbols	This course introduces basic concepts and algorithms in machine learning and neural networks *(the final "." is removed)*
Lowercasing	make every letter lowercase *(also: fix extra spaces & accents)*	**this** course introduces basic concepts and algorithms in machine learning and neural networks *(only "This" changes)*
Tokenization	split into words	this · course · introduces · basic · concepts · and · algorithms · in · machine · learning · and · neural · networks
AFTER tokenization · word by word		
Stopword removal	drop common words	course · introduces · basic · concepts · algorithms · machine · learning · neural · networks · ~~this~~ · ~~and~~ · ~~in~~ · ~~and~~
Stemming	chop off word endings	**cours** · **introduc** · basic · concept · algorithm · **machin** · **learn** · neural · network
Lemmatization	back to dictionary form *(use instead of stemming)*	**course** · **introduce** · basic · concept · algorithm · **machine** · **learning** · neural · network

TOKENIZATION

- **Definition:**
  - Tokens: A fundamental chunk of text that large language models process.
  - Tokenization: splitting text into tokens
- Common words = 1 token; rare words and codes get split (e.g. CS3244 → [CS | 324 | 4])
- **Token Budget**
  - A model can only read a limited number of tokens at once
  - Chunking: cut long text into pieces



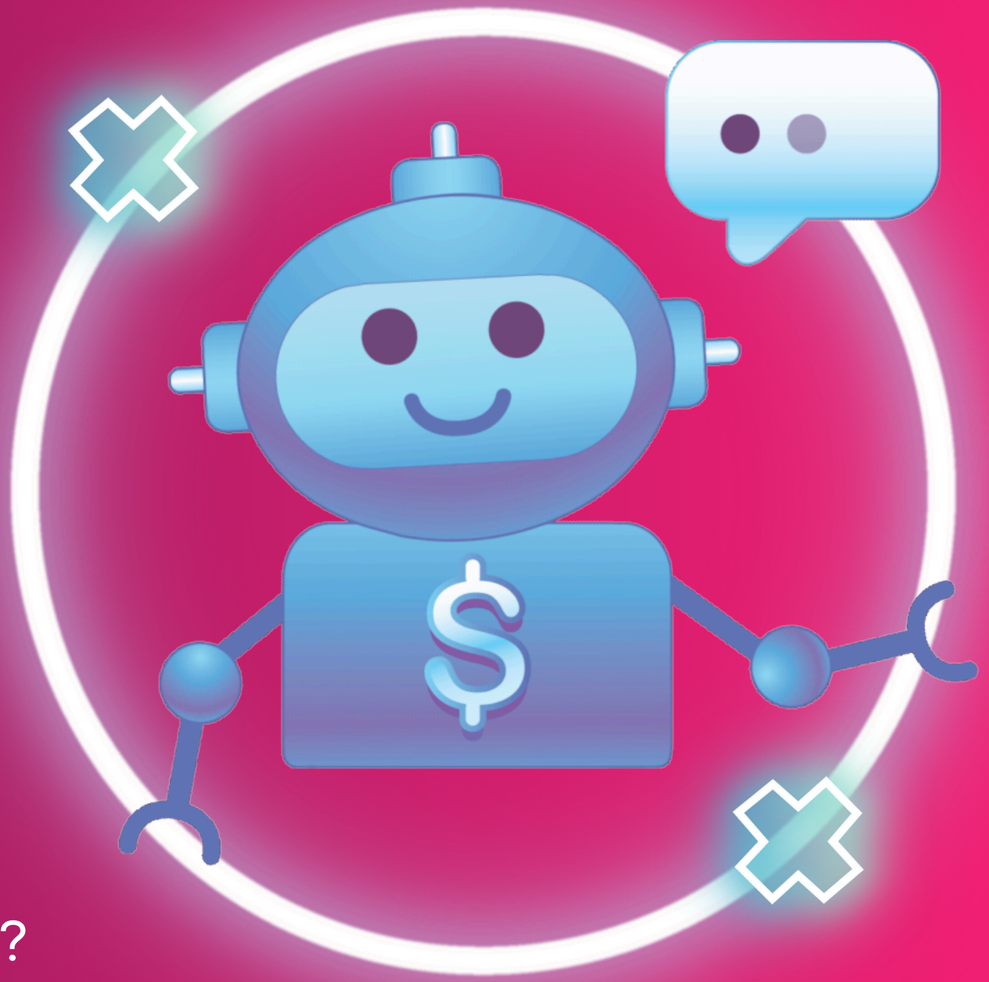
Token count: 24

try it yourself: https://tiktokenizer.vercel.app/

YOUR TURN

Play with the tokenizer code block in our notebook and see how differently all-MiniLM-L6-v2 splits up text (compared with ChatGPT)

https://tiktokenizer.vercel.app/



how does the model decide where to split?

SUBWORD ENCODING

Idea: break word into smaller pieces that can be reused

Split into	"learning" becomes	Pieces to store	Reads new words?	Length
Whole words	learning	✗ too many to store	✗ "NUSMods" → unknown	✓ short
Single letters	l e a r n i n g	✓ very few	✓ can spell anything	✗ way too long
Subwords the sweet spot	learn ing pieces get reused: **learn** + ed, teach + **ing**	✓ a fixed list of pieces	✓ built from known pieces	✓ short enough

- What it does:
  - Common words stay whole: course → [course]
  - Rare words are built from known pieces: CS3244 → [CS | 324 | 4]
  - → The model can read new words with a fixed list of pieces
- Two common methods:
  - Byte-Pair Encoding (BPE)
  - WordPiece

BPE: BYTE-PAIR ENCODING

BPE: Keep gluing together the two pieces that appear side by side most often

	Glue	learning	training	mining
Start	**split into letters**	l e a r n i n g	t r a i n i n g	m i n i n g
Round 1	**i + n → in** 5 times — the most common	l e a r n **in** g	t r a **in** **in** g	m **in** **in** g
Round 2	**in + g → ing** 3 times	l e a r n **ing**	t r a in **ing**	m in **ing**

- New words can still use this splitting method: coding → c | o | d | ing
- mostly used by GPT models

Tokenization, Chunking

WORDPIECE

WordPiece: BPE's cousin

- Same idea: start from letters, glue pieces together
- Difference: BPE glues the most common pair; WordPiece glues the pair that sticks together most (pieces that rarely show up apart)
- Pieces inside a word start with ##: GPU → [gp | ##u]

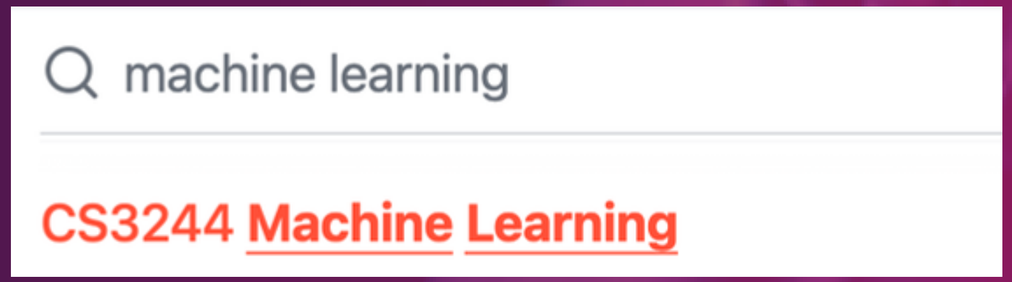
Pair of tokens	Frequency together	Frequency of first token	Frequency of second token
a + b	20	100	100
x + y	10	10	10

Feature Extraction

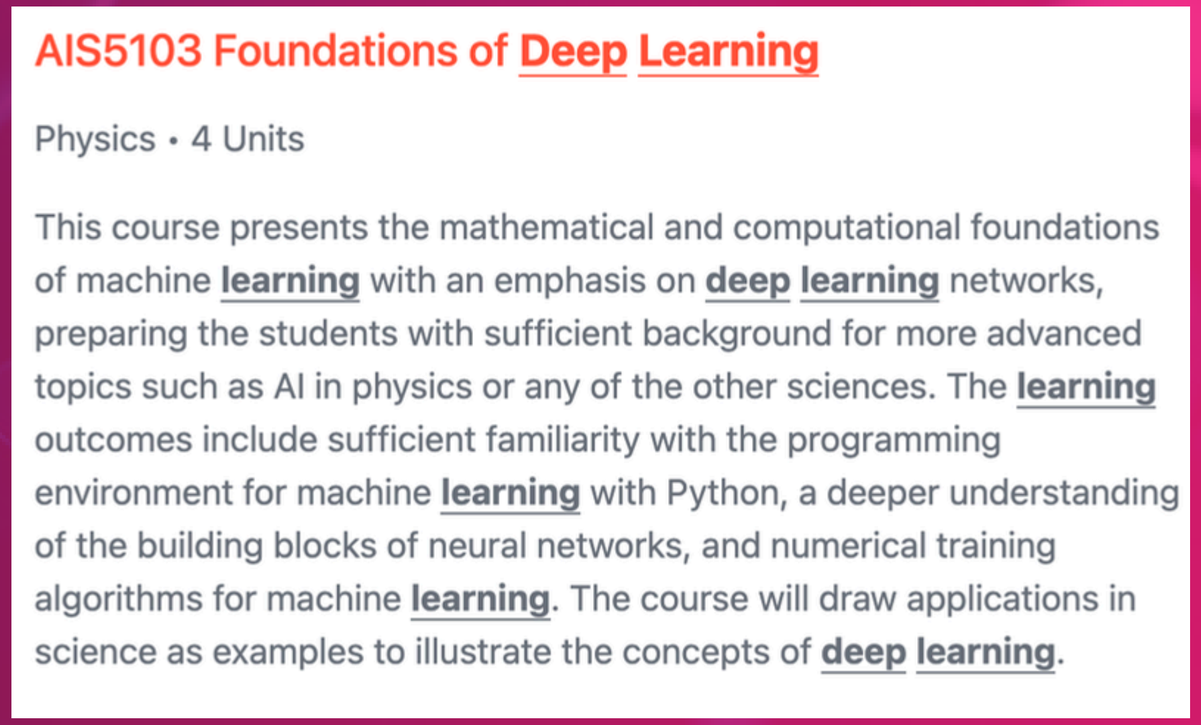
THE SEARCH PROCESS

- With words turned into numbers, how can we search for courses?
  - Keyword search: returns titles that have the exact match
  - Exact, but ignores similar titles we might want :(

Search “machine learning”:



But not including:



Idea: look at all words in the description, not just the title itself

Feature Extraction

CURRENT METHODS - BAG OF WORDS

BoW: Count each word in a description → text becomes a row of numbers

	course	covers	machine	deep	learning	economics
“course covers machine learning”	1	1	1	0	1	0
“course covers deep learning”	1	1	0	1	1	0
“course covers economics”	1	1	0	0	0	1

Search process: count the shared words

e.g. Query “machine learning course” gives a count of 3 / 2 / 1 for each entry

- Problems:
  - Every word counts the same: “course” is in every description, yet it still gives the economics course a point
- Order is lost: “students teach tutors” = “tutors teach students”

Feature Extraction

CURRENT METHODS - TF-IDF

TF-IDF: Term Frequency–Inverse Document Frequency

- TF (term frequency): how often the word appears in this description
- IDF (inverse document frequency): how rare the word is across all descriptions
- TF-IDF = TF × IDF → frequent here + rare elsewhere = important

"course covers machine learning, deep learning"	course	covers	machine	learning ×2	deep
Bag of Words count in this description	1	1	1	2	1
TF count ÷ 6 words in this description	1/6 = **0.17**	1/6 = **0.17**	1/6 = **0.17**	2/6 = **0.33**	1/6 = **0.17**
IDF $\log_{10}$(3 ÷ descriptions that have the word)	log(3/3) = **0**	log(3/3) = **0**	log(3/1) = **0.477**	log(3/2) = **0.176**	log(3/2) = **0.176**
TF-IDF TF × IDF	0.17 × 0 = **0**	0.17 × 0 = **0**	0.17 × 0.477 ≈ **0.08**	0.33 × 0.176 ≈ **0.06**	0.17 × 0.176 ≈ **0.03**

"learning" vs "deep": same IDF, but "learning" appears twice → TF doubles → TF-IDF doubles (0.06 vs 0.03)

3 descriptions: "course covers machine learning, deep learning" · "course covers deep learning" · "course covers economics"

Feature Extraction

CURRENT METHODS - BM25

BM25: Best Matching 25

- Combines TF-IDF and BoW
  - Adjusts scores relative to the average document length to prevent long documents from dominating rankings just because they contain more words
- Saturation: the 10th “learning” adds much less than the 1st
- Length: long descriptions don't win just by having more words

Feature Extraction

CURRENT METHODS - BM25

BM25: Best Matching 25

"course covers machine learning, deep learning"	course	covers	machine	learning ×2	deep
Bag of Words count in this description	1	1	1	2	1
TF count ÷ 6 words in this description	1/6 = **0.17**	1/6 = **0.17**	1/6 = **0.17**	2/6 = **0.33**	1/6 = **0.17**
IDF $\log_{10}$(3 ÷ descriptions that have the word)	log(3/3) = **0**	log(3/3) = **0**	log(3/1) = **0.477**	log(3/2) = **0.176**	log(3/2) = **0.176**
TF-IDF TF × IDF	0.17 × 0 = **0**	0.17 × 0 = **0**	0.17 × 0.477 ≈ **0.08**	0.33 × 0.176 ≈ **0.06**	0.17 × 0.176 ≈ **0.03**
BM25 TF count with diminishing returns, adjusted for length	count 1 → **0.86**	count 1 → **0.86**	count 1 → **0.86**	count 2 → **1.24**	count 1 → **0.86**
BM25 IDF × BM25 TF	0 × 0.86 = **0**	0 × 0.86 = **0**	0.477 × 0.86 ≈ **0.41**	0.176 × 1.24 ≈ **0.22**	0.176 × 0.86 ≈ **0.15**

"learning" vs "deep": same IDF, "learning" appears twice → TF-IDF doubles (0.06 vs 0.03), BM25 only ×1.4 (0.22 vs 0.15)

3 descriptions: "course covers machine learning, deep learning" · "course covers deep learning" · "course covers economics" | **BM25:** k1 = 1.2, b = 0.75

HALLUCINATION

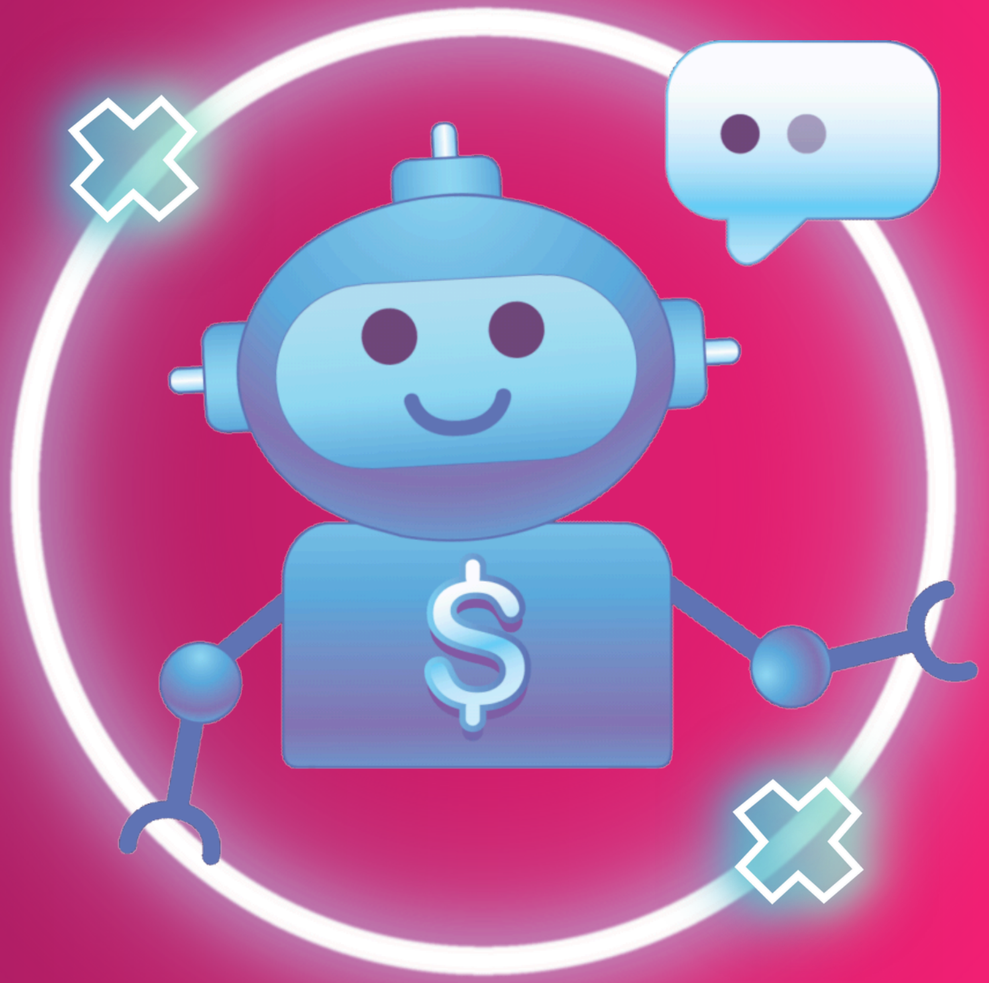
Background

- When we use an LLM to search something, it might:
  - not know the updated data
  - invent fluent but untrue facts
- Reasons:
  - Knowledge cutoff: it only knows what it was trained on, up to a certain date
  - Next-token prediction: it picks what sounds likely, not what is true

✗ **Without retrieval**	✓ **With retrieved NUSMods data**
LLM's answer: confident, but outdated or wrong	Answer based on the CS3244 entry Source: NUSMods

YOUR TURN

Ask Gemini something about NUSMods, and then verify the answer with real NUSMods data



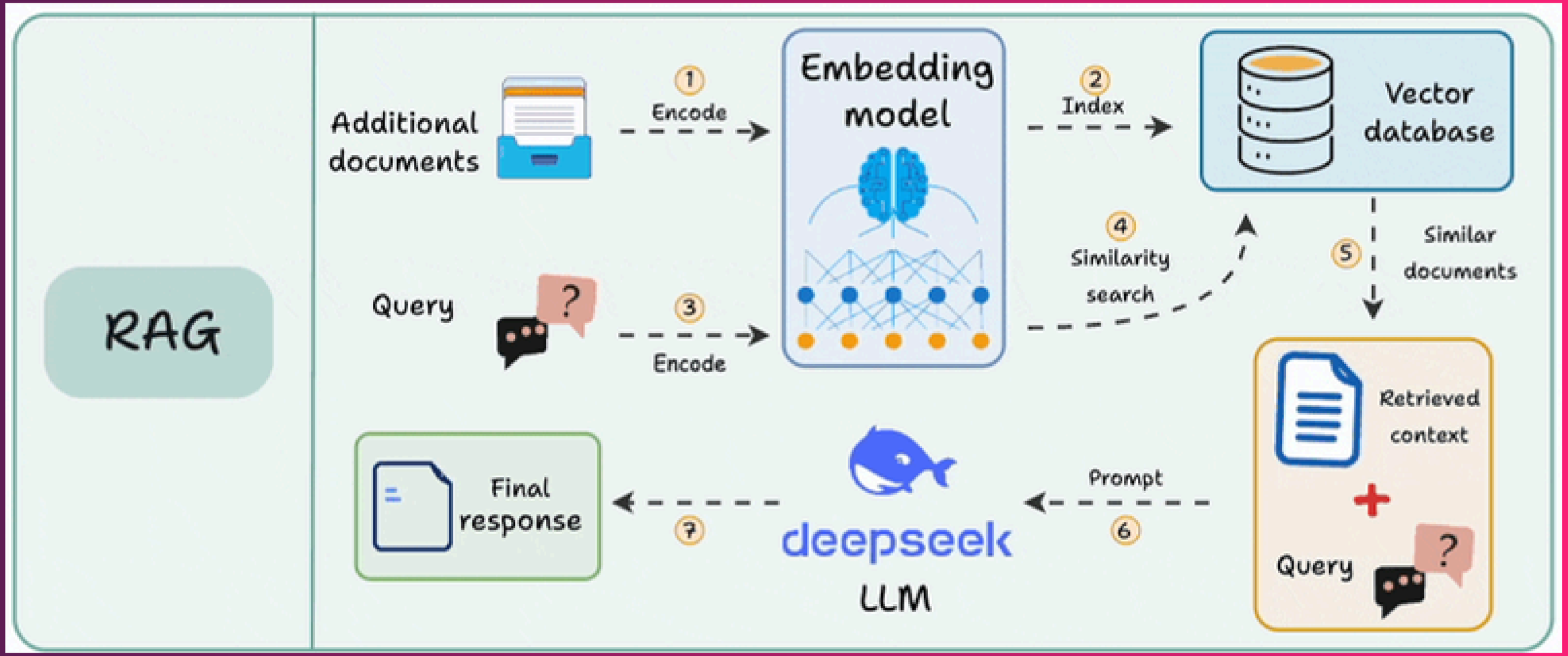
THE CONCEPT OF RAG

DEFINITION

- Retrieval-Augmented Generation (RAG)
- A framework that improves Large Language Models (LLMs) by connecting them to external knowledge sources

Why not directly put the information as prompt?

- Token budget: NUSMods has too many descriptions to fit into a single prompt
- Solution: retrieve the most relevant


RAG
Additional documents
1
Encode
Embedding model
2
Index
Vector database
Query
3
Encode
4
Similarity search
5
Similar documents
Retrieved context
+
Query
Prompt
6
deepseek
LLM
7
Final response


BEFORE tokenization · on the whole text
AFTER tokenization · word by word
1 Raw text
NUSMods description
2 Cleaning
remove junk & symbols
3 Normalization
lowercase, fix spacing & accents
4 Tokenization
split into tokens
5 Normalization
remove stopwords, stem / lemmatize
6 Chunking
cut long text to fit the token budget
7 Feature extraction
Bag of Words, TF-IDF, BM25
8 Embedding
text → meaning vectors (MiniLM)
9 Retrieval
find the closest matches

RECAP

- So far: text → tokens → word counts (Bag of Words, TF-IDF, BM25) → find the best-matching courses → give them to the LLM (RAG)

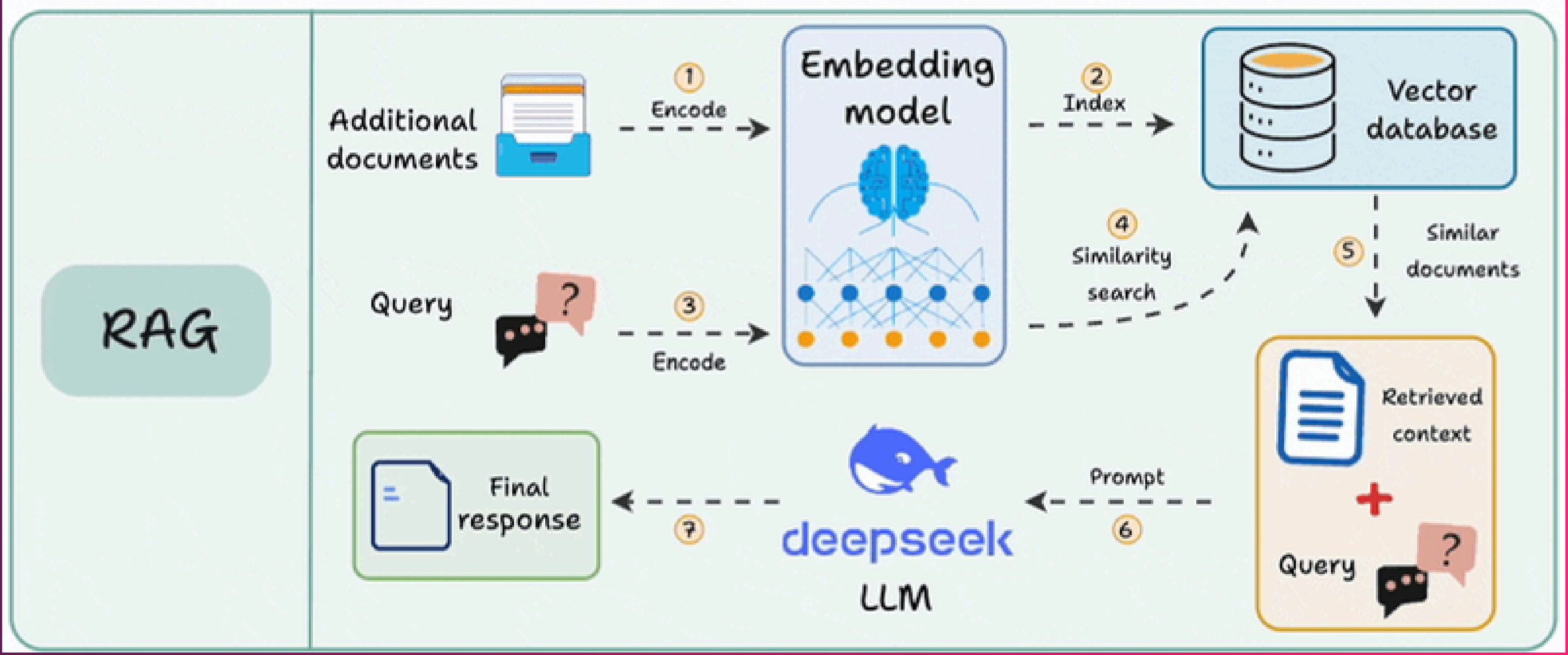
- But keyword methods only match the exact same words:
  - Search: “artificial intelligence”
  - CS3244: “...basic concepts and algorithms in machine learning and neural networks.”
  - 0 shared words → BM25 score = 0, even though it's clearly an AI course

- Word count still does not capture meaning, which is why we need embeddings.

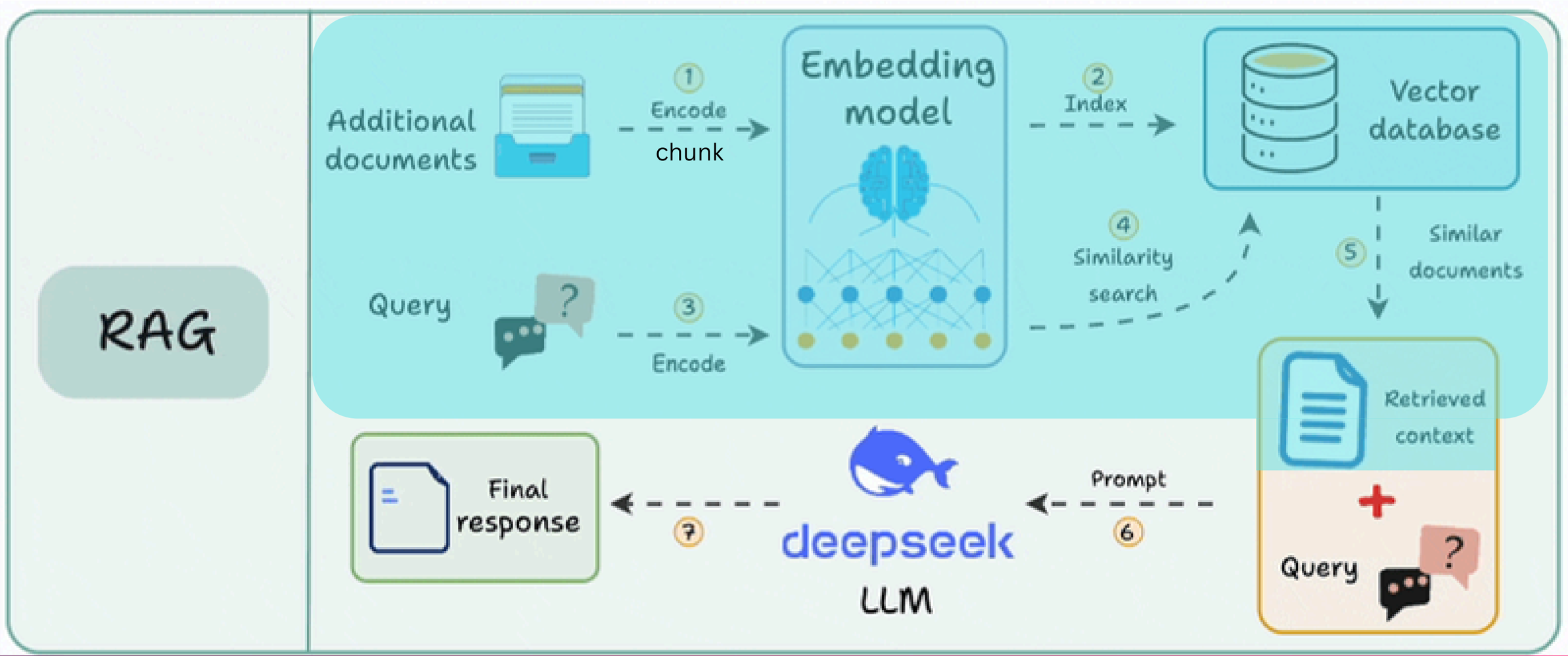


RETRIEVAL ENGINE

Remember this?



Retrieval Engine



Word counts don’t capture meaning

Search
“Artificial Intelligence”

CS3244 - Machine Learning
“…basic concepts and algorithms in machine learning and neural networks.”

Shared words
0

BM25 score
0

CS3244 is clearly related to AI, but the search can’t tell.

Sparse vectors vs dense vectors

BOW · TF-IDF · BM25

Sparse: one column per word

	artificial	intelligence	machine	neural
query	1	1	0	0
CS3244	0	0	1	1

...and thousands more columns, almost all 0. No column where both are non-zero, so the score is 0.

EMBEDDING

Dense

`[0.12, -0.41, …, 0.33]`

- 384 numbers, all filled in
- Dimensions learned by a model
- Encodes what the text is about

We want “artificial intelligence” to land close to CS3244, even with **no shared words**.



Meet the data: one NUSMods Record


```
{
  "moduleCode": "CS3244",
  "title": "Machine Learning",
  "description": "This course introduces basic
concepts and algorithms in machine learning and neural
networks. …",
  "workload": [ … ],
  … and more fields
}
```

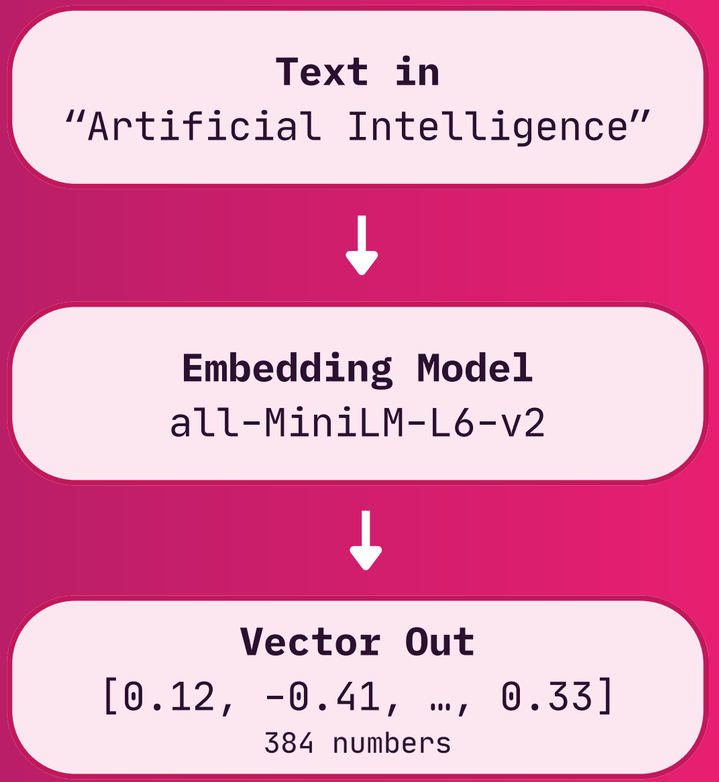

6552 Modules: AY 26/27



Embedding

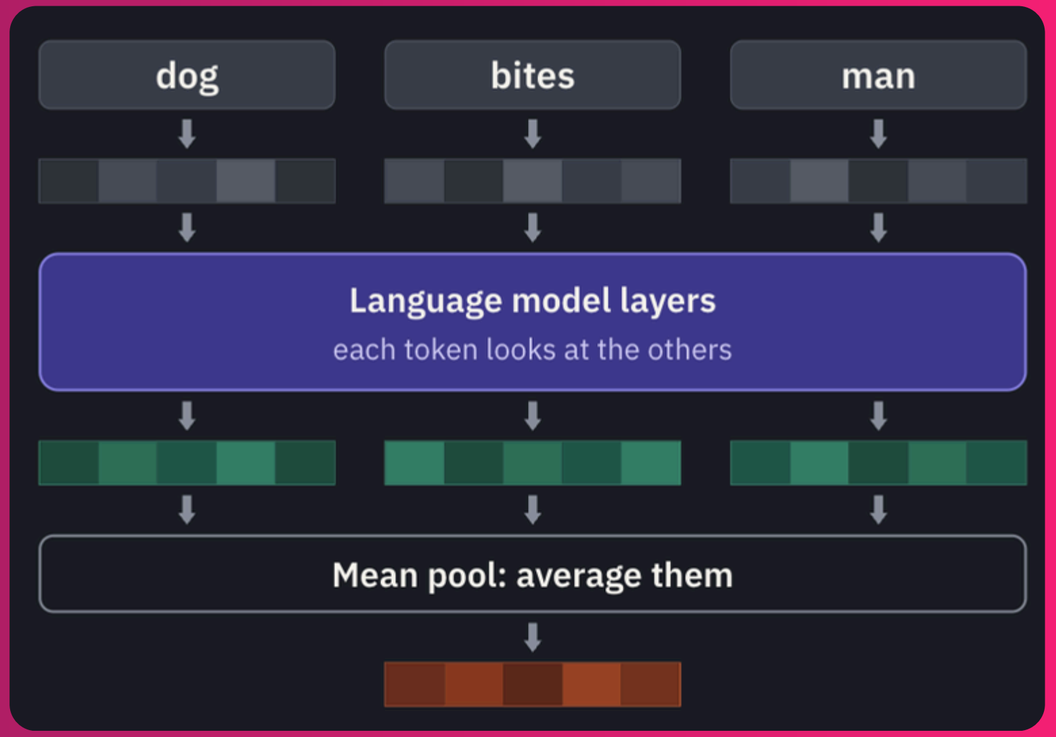
An embedding turns text into a list of numbers, called a vector.

A two-word query and a whole module description both come out as **384 numbers**. Same length every time, so we can compare anything with anything.

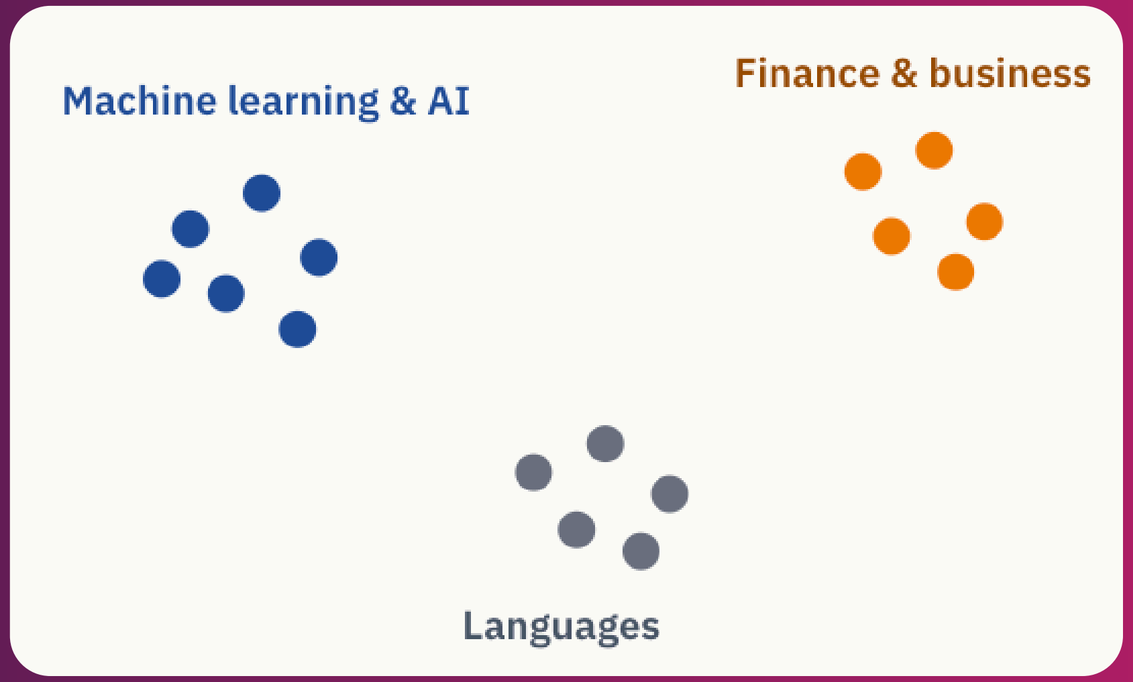


How it works

1. Tokenize
    a. Tokenizer will split the texts into tokens
2. Lookup
    a. Each token gets a starting vector. Same token will get same vector every time.
3. Language Model Layers
    a. Tokens look at each other, so “bites” knows who is biting whom.
4. Mean Pooling
    a. Average the token vectors into one: 384 numbers for the whole text.



Meaning becomes Position

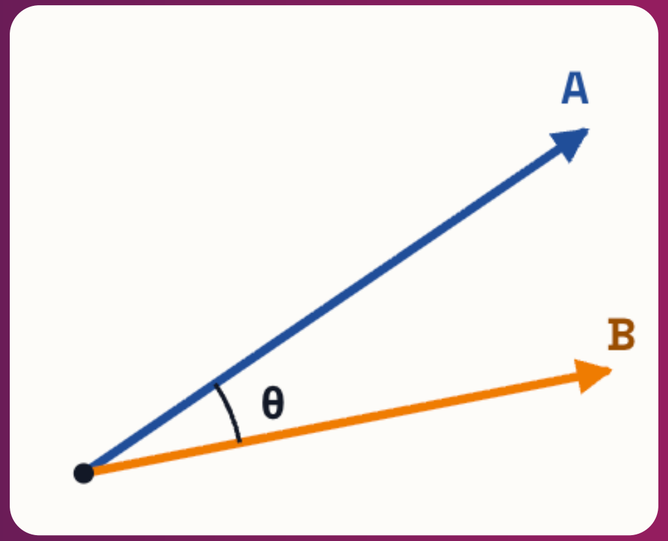


Related texts/modules will tend to be quite close

Unrelated texts/modules will be far apart

Real embeddings have hundreds of dimensions. We squash them to 2D just so we can look at them.

Measuring closeness: cosine similarity

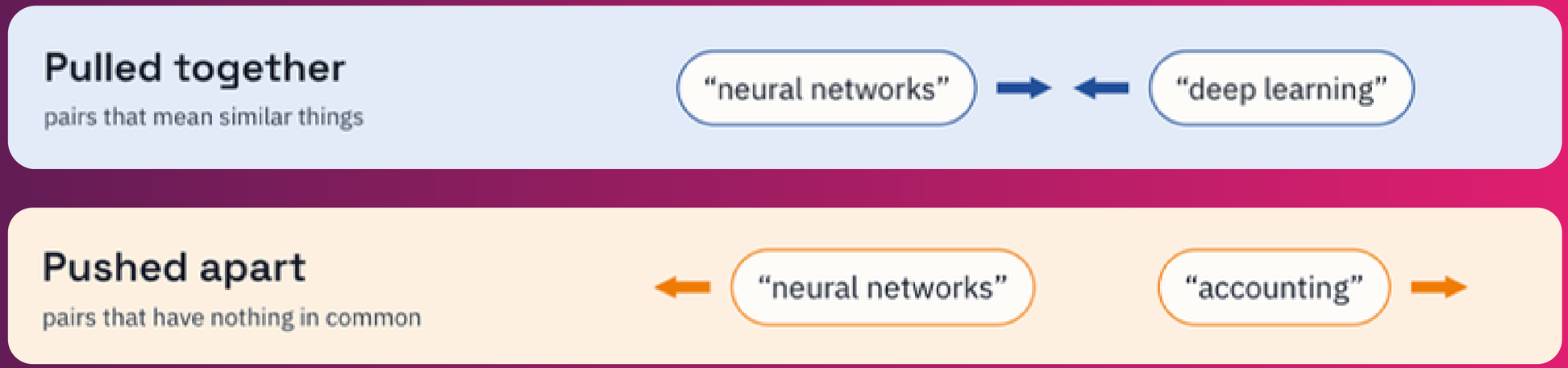


Angle	Score	Means
Small	close to 1	very related
90°	0	unrelated
Opposite	−1	opposite (rare for text)

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$$

Only the direction counts, not the length of the arrow.

How the model learns this similarity



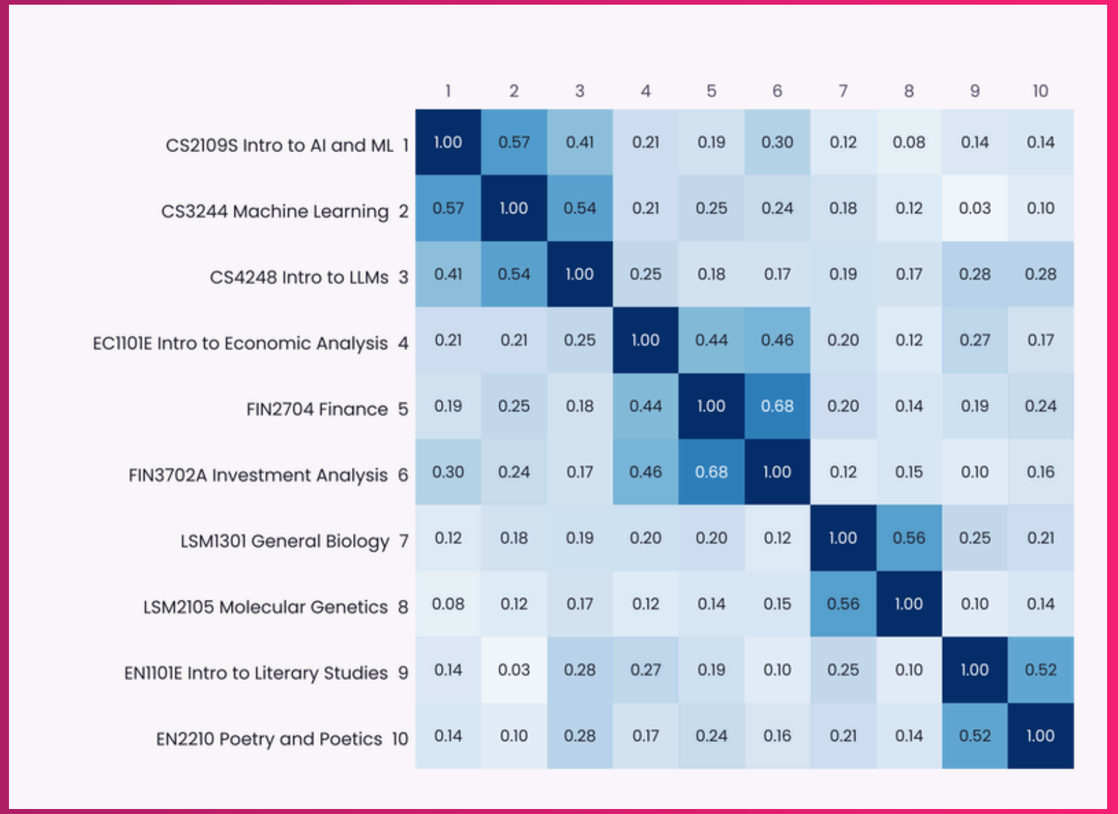
Repeat over a huge number of text pairs, and related meanings end up close together.

YOUR TURN

Similarity heatmap

1. Run the heatmap cell for ~10 modules across faculties
2. Edit the sentence list: add your own modules or sentences
3. Re-run and compare the pairs

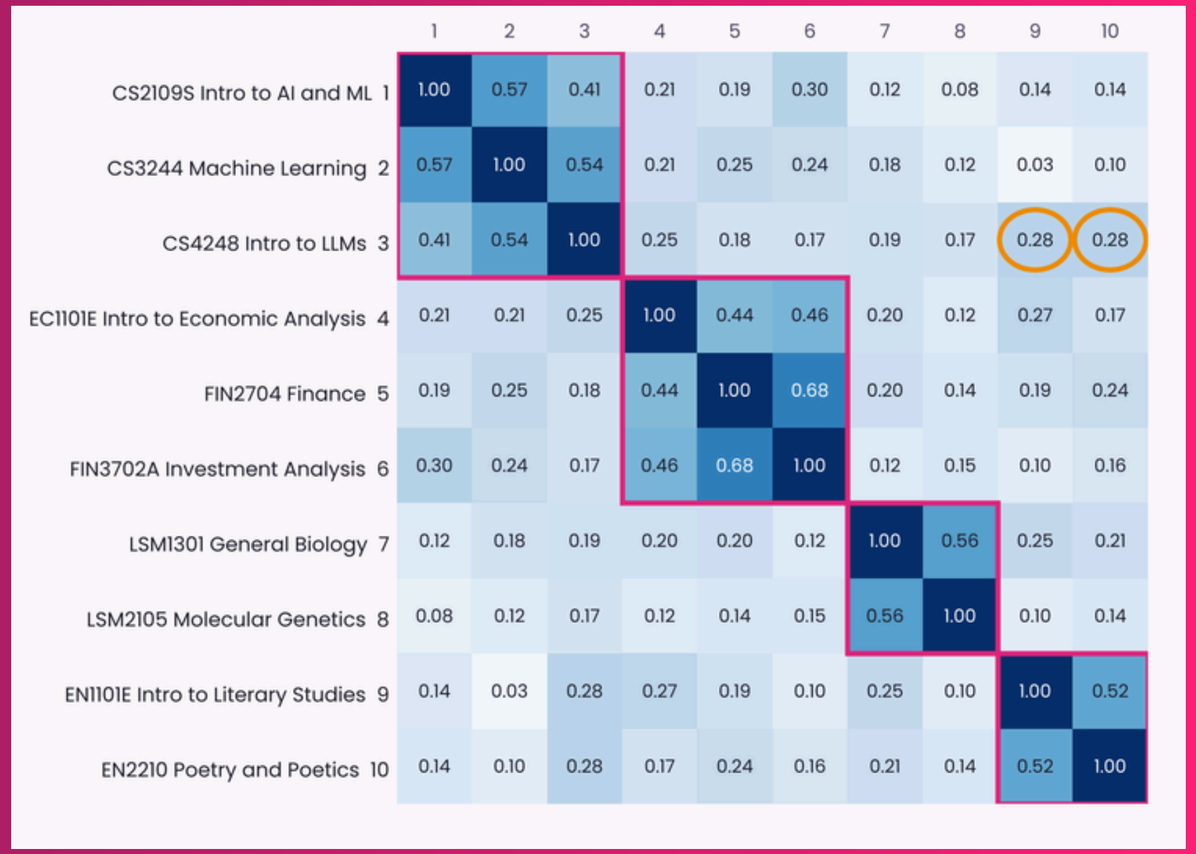
Which pairs surprise you?



	1	2	3	4	5	6	7	8	9	10
CS2109S Intro to AI and ML 1	1.00	0.57	0.41	0.21	0.19	0.30	0.12	0.08	0.14	0.14
CS3244 Machine Learning 2	0.57	1.00	0.54	0.21	0.25	0.24	0.18	0.12	0.03	0.10
CS4248 Intro to LLMs 3	0.41	0.54	1.00	0.25	0.18	0.17	0.19	0.17	0.28	0.28
EC1101E Intro to Economic Analysis 4	0.21	0.21	0.25	1.00	0.44	0.46	0.20	0.12	0.27	0.17
FIN2704 Finance 5	0.19	0.25	0.18	0.44	1.00	0.68	0.20	0.14	0.19	0.24
FIN3702A Investment Analysis 6	0.30	0.24	0.17	0.46	0.68	1.00	0.12	0.15	0.10	0.16
LSM1301 General Biology 7	0.12	0.18	0.19	0.20	0.20	0.12	1.00	0.56	0.25	0.21
LSM2105 Molecular Genetics 8	0.08	0.12	0.17	0.12	0.14	0.15	0.56	1.00	0.10	0.14
EN1101E Intro to Literary Studies 9	0.14	0.03	0.28	0.27	0.19	0.10	0.25	0.10	1.00	0.52
EN2210 Poetry and Poetics 10	0.14	0.10	0.28	0.17	0.24	0.16	0.21	0.14	0.52	1.00

Result Example

- Pink boxes = same area
- diagonal = 1, comparing the module with itself
- Unrelated ≈ 0.1 to 0.3, not 0
- CS4248 (LLMs) scores 0.28 with both literature modules, more than with Finance (0.18). Both are about language.

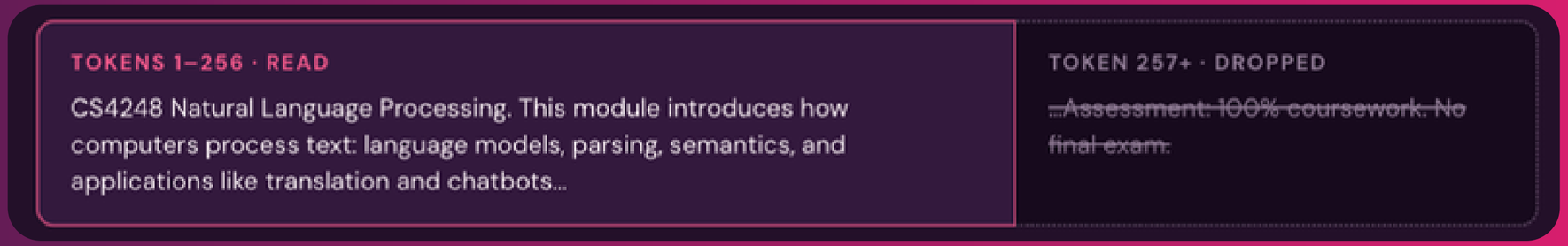


	1	2	3	4	5	6	7	8	9	10
CS2109S Intro to AI and ML 1	1.00	0.57	0.41	0.21	0.19	0.30	0.12	0.08	0.14	0.14
CS3244 Machine Learning 2	0.57	1.00	0.54	0.21	0.25	0.24	0.18	0.12	0.03	0.10
CS4248 Intro to LLMs 3	0.41	0.54	1.00	0.25	0.18	0.17	0.19	0.17	0.28	0.28
EC1101E Intro to Economic Analysis 4	0.21	0.21	0.25	1.00	0.44	0.46	0.20	0.12	0.27	0.17
FIN2704 Finance 5	0.19	0.25	0.18	0.44	1.00	0.68	0.20	0.14	0.19	0.24
FIN3702A Investment Analysis 6	0.30	0.24	0.17	0.46	0.68	1.00	0.12	0.15	0.10	0.16
LSM1301 General Biology 7	0.12	0.18	0.19	0.20	0.20	0.12	1.00	0.56	0.25	0.21
LSM2105 Molecular Genetics 8	0.08	0.12	0.17	0.12	0.14	0.15	0.56	1.00	0.10	0.14
EN1101E Intro to Literary Studies 9	0.14	0.03	0.28	0.27	0.19	0.10	0.25	0.10	1.00	0.52
EN2210 Poetry and Poetics 10	0.14	0.10	0.28	0.17	0.24	0.16	0.21	0.14	0.52	1.00

Chunking: Why split text into chunks?

Every model has a token budget

Each embedding model can only read so many tokens. Past the limit, text is silently dropped.



One giant vector means nothing in particular

Squash every module into one vector and it lands in the middle of everything, close to no query in particular.

So, how big should the chunks be?

Which chunk size would you pick?

A

One topic group per chunk

> “AI modules: CS3244 Machine Learning. ... CS4248 Natural Language Processing. ... AIS5103 Foundations of Deep Learning. ... + more”

B

One sentence per chunk

> “Students will also complete a group project.”

C

One module per chunk

> “CS3244 Machine Learning. This course introduces basic concepts and algorithms in machine learning and neural networks. ...”

In your opinion, which chunk is the best one?



Which chunk size would you pick? → C

A

One topic group per chunk

> “AI modules: CS3244 Machine Learning. ... CS4248 Natural Language Processing. ... AIS5103 Foundations of Deep Learning. ... + more”

Too Big, Overbudget

- “Which module teaches NLP?” You get the whole AI chunk back, not CS4248.
- A few descriptions already pass 256 tokens, so the rest of the group gets cut.

B

One sentence per chunk

> “Students will also complete a group project.”

Too Small, Lost Context

- “Students will also complete a group project.” Which module? Could be half of NUSMods.
- Accurate, but useless on its own.

C

One module per chunk

> “CS3244 Machine Learning. This course introduces basic concepts and algorithms in machine learning and neural networks. ...”

The best one

- It fits the token budget, it says which module it is, and follows the natural boundary of data.

Different ways to chunk



Fixed size

Cut every N tokens.

Simple, but can split a sentence in half.



Fixed size + overlap

Each chunk repeats the end of the one before (dark bits).

Less context lost at the cuts, but more chunks to store.



Natural boundaries

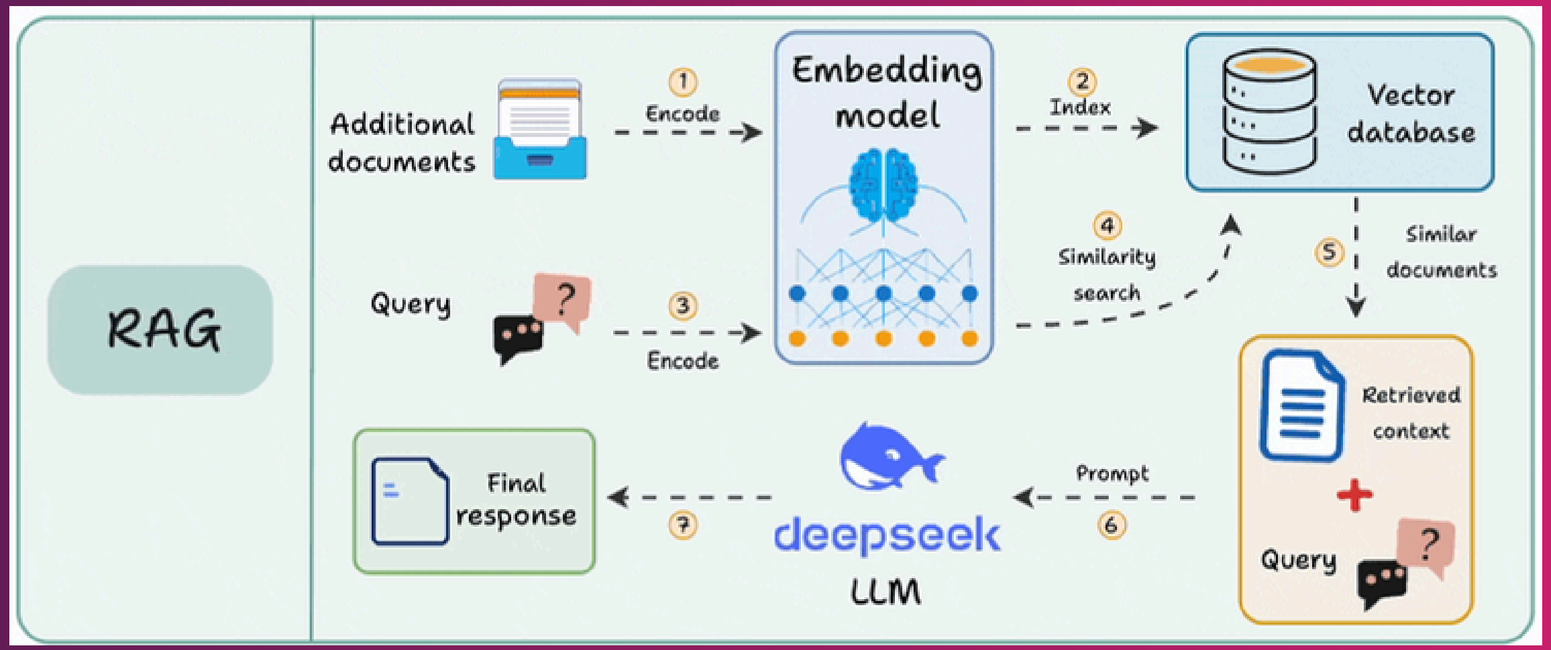
Split where the data already splits: modules, paragraphs, sections.

Keeps meaning whole. Needs structured data.

What we use

NUSMods already comes split by module, so one module will always be one chunk.

Pipeline



Step 1-2: Indexing Pipeline
Done once at the start

Step 3-5: Retrieval Pipeline
Query time: every search

Step 6-7: Generation Pipeline
In the next section!

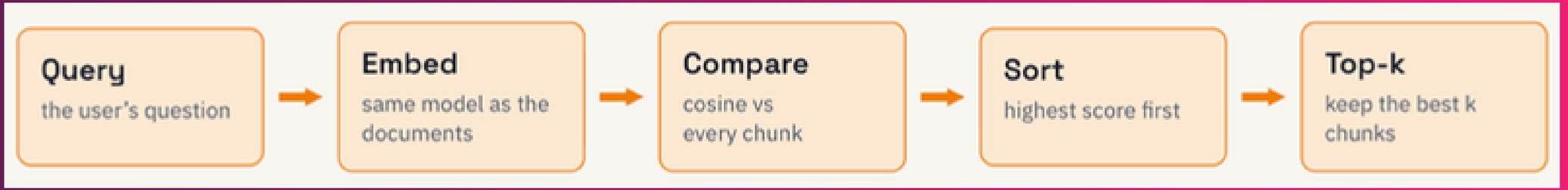
Indexing Pipeline



The pipeline will be done once at the start. Every module goes through this once. At search time, only the query gets embedded.

Store usually means a vector database (Chroma, FAISS, Pinecone...). However, for today it's just the embeddings array from setup saved locally.

Query Pipeline



We need to use the same embedding model for queries and documents. A different model puts vectors in a different space, and the scores might not be accurate.

Did embedding fix the problem?

Now, let's try to search modules using "artificial intelligence" and see if CS3244 (Machine Learning) comes up or not.

BM25

#1 IT1244 AI: Technology and Impact · 4.73

#4 UTS2104 Intelligence and Singapore Society: about human intelligence, not AI

CS3244: score 0, tied with 6,344 other modules at the bottom

Embeddings (cosine)

#1 IT5005 Artificial Intelligence · 0.60

Nearly every module above CS3244 is about AI.

CS3244: score 0.44, rank 51 of 6,552 (top 1%)

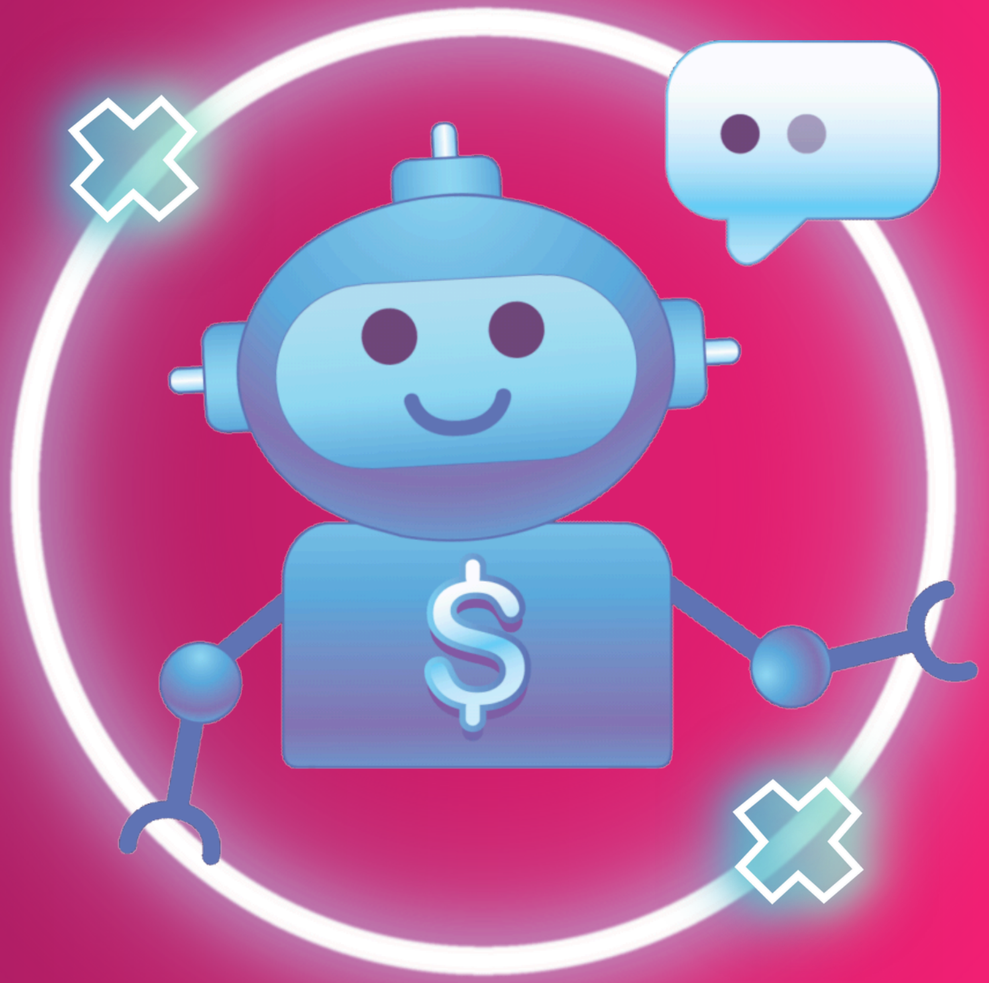
Not #1 answer, but from a tie at the bottom into the top 1%, and no longer fooled by the word "intelligence".



YOUR TURN

Ask your own questions to the engine!

See whether it give you the right chunks



Limitations

Query	Top Result	What went wrong
hi	HY6770 Graduate Research Seminar (0.29)	Nothing relevant, still returns k results
CS2040S prereqs	CS3245R Information Retrieval (0.34)	Codes carry no meaning
modules without final exam	MA3201 Algebra II	Negation gets ignored
lightest workload module	MT5666 Industrial Attachment	Can’t compare numbers
pembelajaran mesin	LAB3202 Bahasa Indonesia 4	Trained mostly on English

Exact Matches
Combine with BM25: hybrid search

Junk Results
Generation must know when to say “I don’t know”

EVALUATION

RAG evaluation is the process of measuring how well a Retrieval-Augmented Generation system retrieves relevant information and uses it to generate accurate, useful answers.
Why do we evaluate RAG?

Assess retrieval quality: Does the system find the right documents for the user's question?

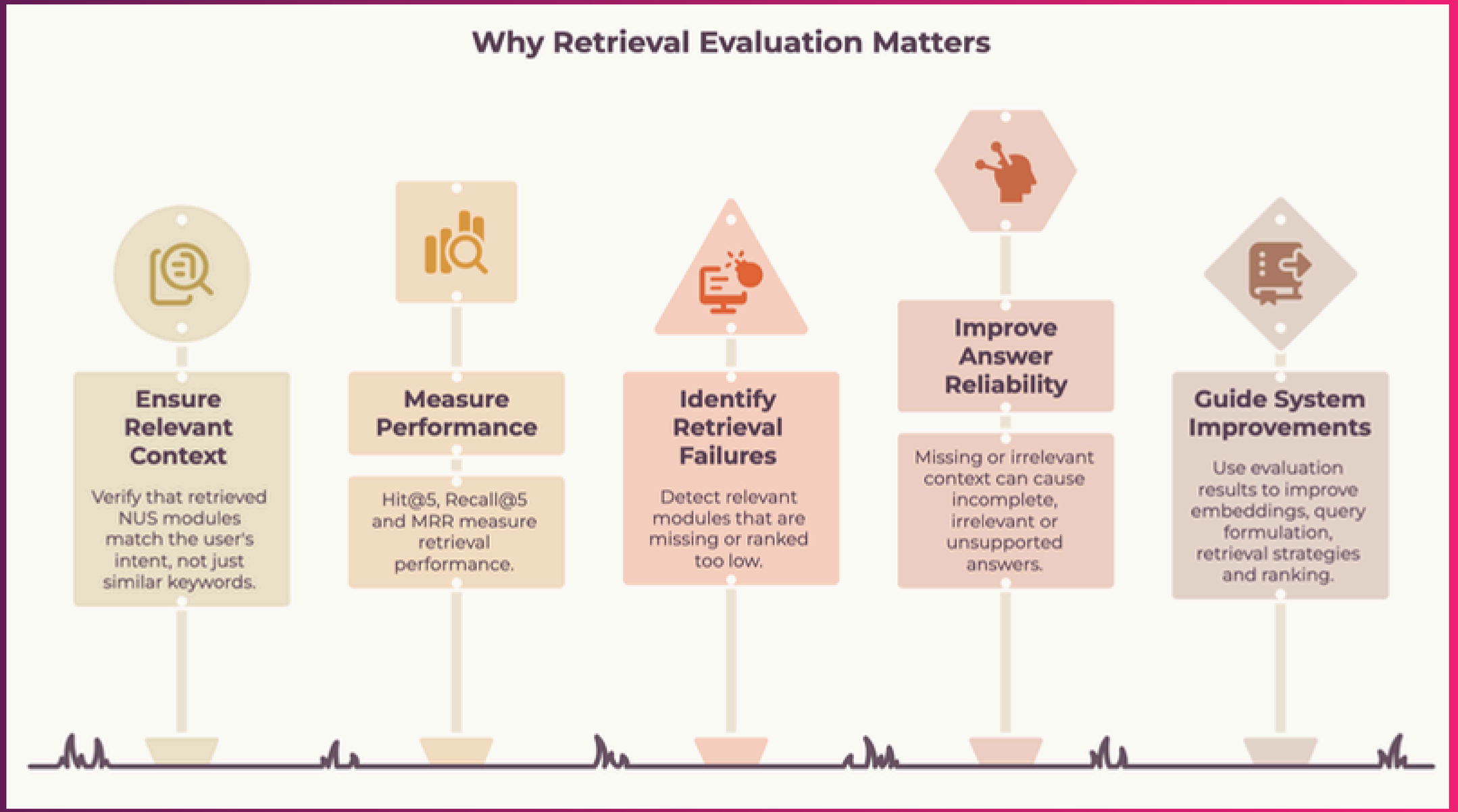
Assess answer quality: Does the LLM generate an accurate, relevant answer based on the retrieved context?

Identify errors: Determine whether problems come from retrieving the wrong information or generating an incorrect answer.

Improve reliability: Use evaluation results to refine the retrieval process and overall RAG pipeline.



Why Evaluate Retrieval?



Evaluation Metrics

1. Hit@5 — Retrieval Success

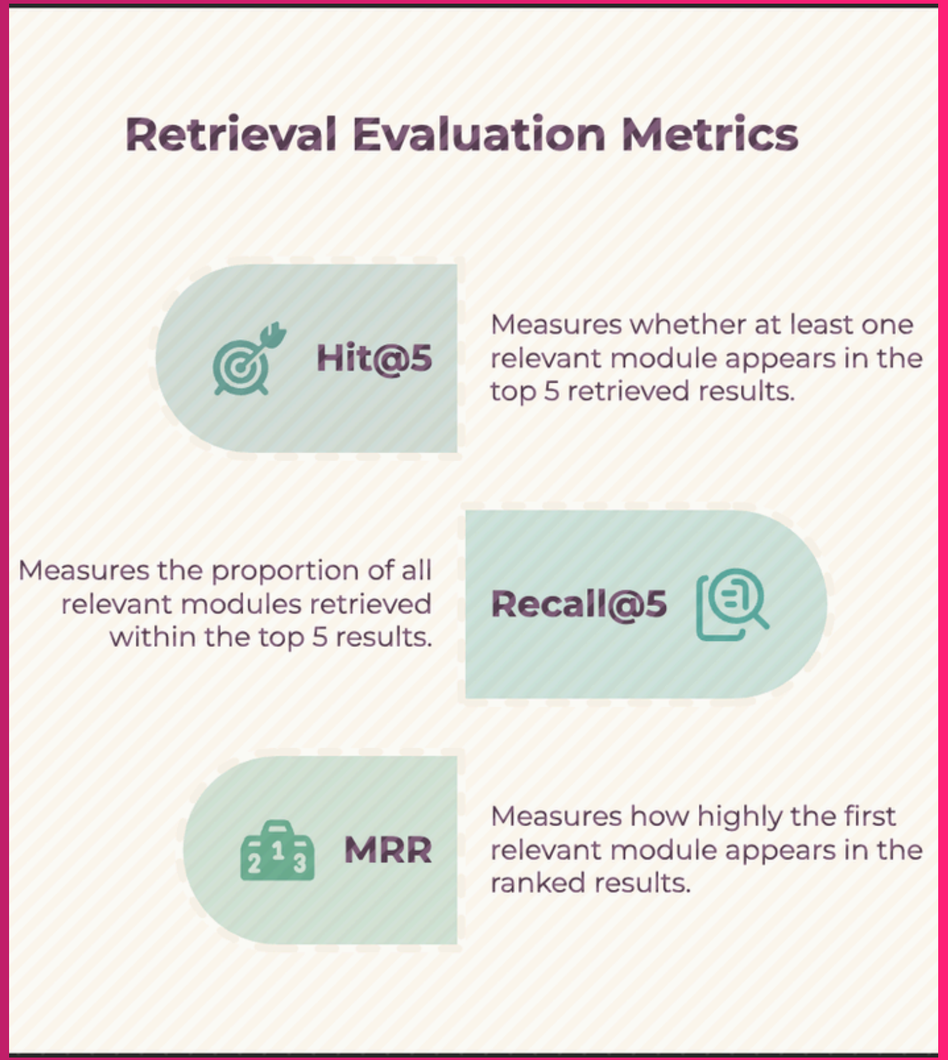
Measures whether at least one relevant module appears in the top 5 retrieved results.

2. Recall@5 — Relevant Information Coverage

Measures the proportion of all relevant modules retrieved within the top 5 results.

3. Mean Reciprocal Rank (MRR)— Ranking Quality

Measures how highly the first relevant module appears in the ranked results.



HIT@5

Definition: Measures whether the retrieval system retrieves at least one relevant document within the top 5 results.
Formula:

$$Hit@5 = \begin{cases} 1 & \text{if a relevant document is in top 5} \\ 0 & \text{otherwise} \end{cases}$$

Made with Napkin

Example:
Retrieved documents: D1, D2, D3, D4, D5

If D3 is relevant, Hit@5 = 1.

If none of the five documents is relevant, Hit@5 = 0.

RECALL@5

Definition: Measures the proportion of all relevant documents that are successfully retrieved within the top 5 results.

Formula:

$$Recall@5 = \frac{\text{Relevant documents in top 5}}{\text{Total relevant documents}}$$

Example:

- Total relevant documents = 10
- Relevant documents retrieved in top 5 = 4

$$Recall@5 = \frac{4}{10} = 0.40$$

Mean Reciprocal Rank (MRR)

Definition: Measures how highly the first relevant document appears in the retrieved results.
Formula:

$$MRR = \frac{1}{N} \sum_{i=1}^{N} \frac{1}{rank_i}$$

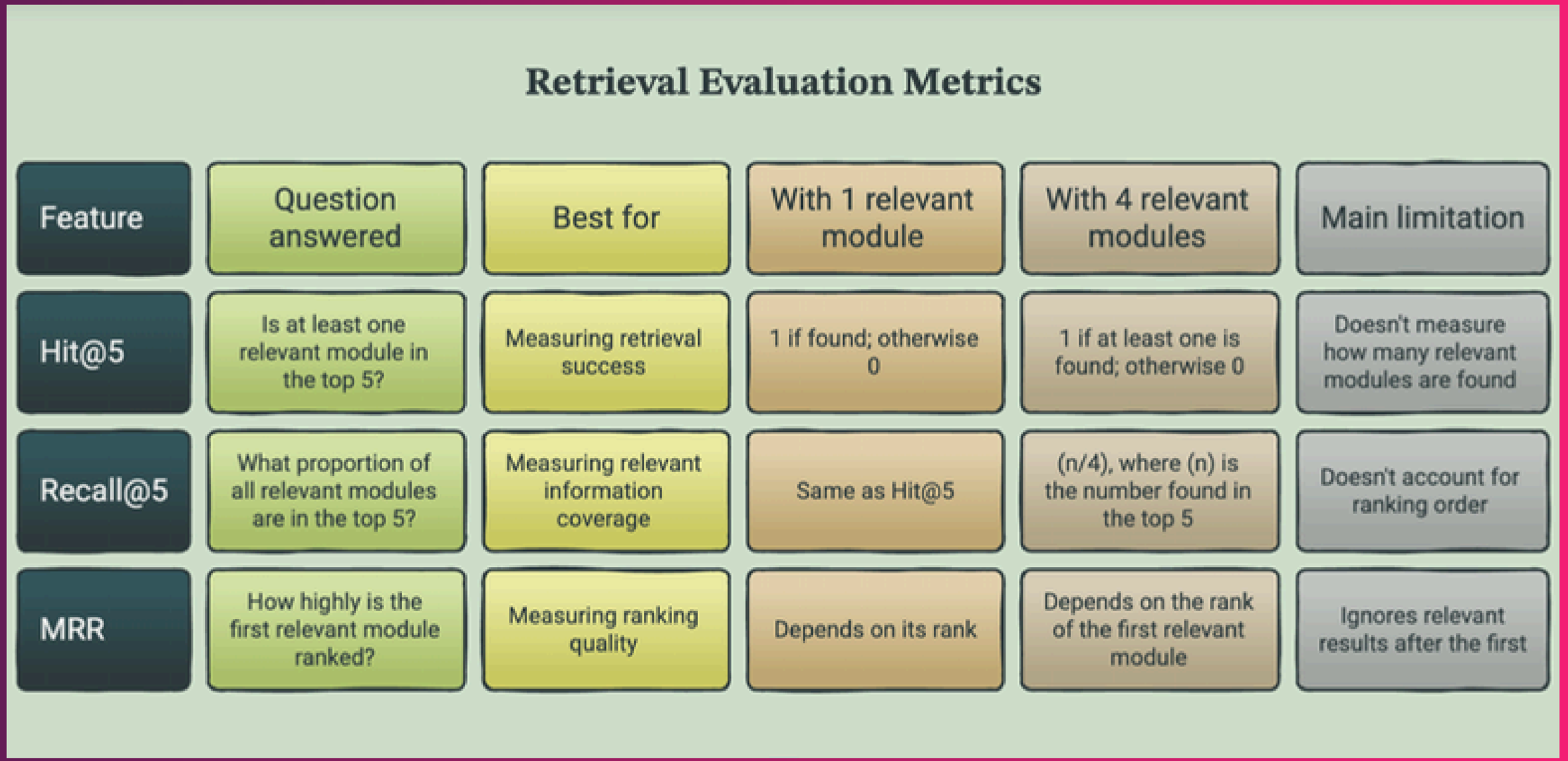
Example:

Query	First relevant document rank	Reciprocal rank
Q1	1	1.00
Q2	2	0.50
Q3	4	0.25

$$MRR = \frac{1 + 0.5 + 0.25}{3} = 0.583$$

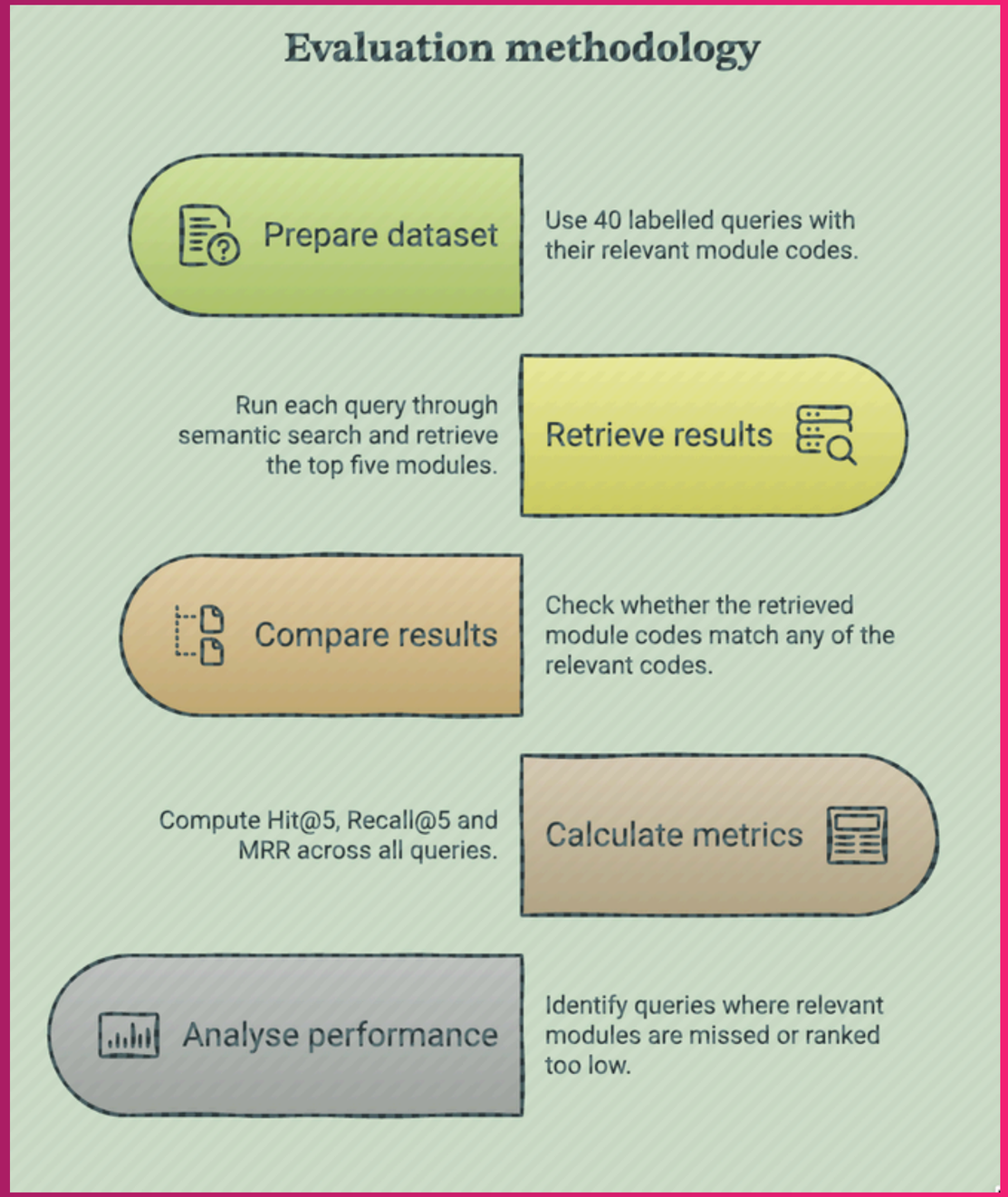
Made with Napkin

Metrics Comparison



Evaluation Methodology

- Prepare the evaluation dataset: Use 40 labelled queries with their relevant module codes.
- Retrieve results: Run each query through semantic search and retrieve the top five modules.
- Compare results: Check whether the retrieved module codes match any of the relevant codes.
- Calculate metrics: Compute Recall@5, Hit@5 and MRR across all queries.
- Analyse performance: Identify queries where relevant modules are missed or ranked too low.



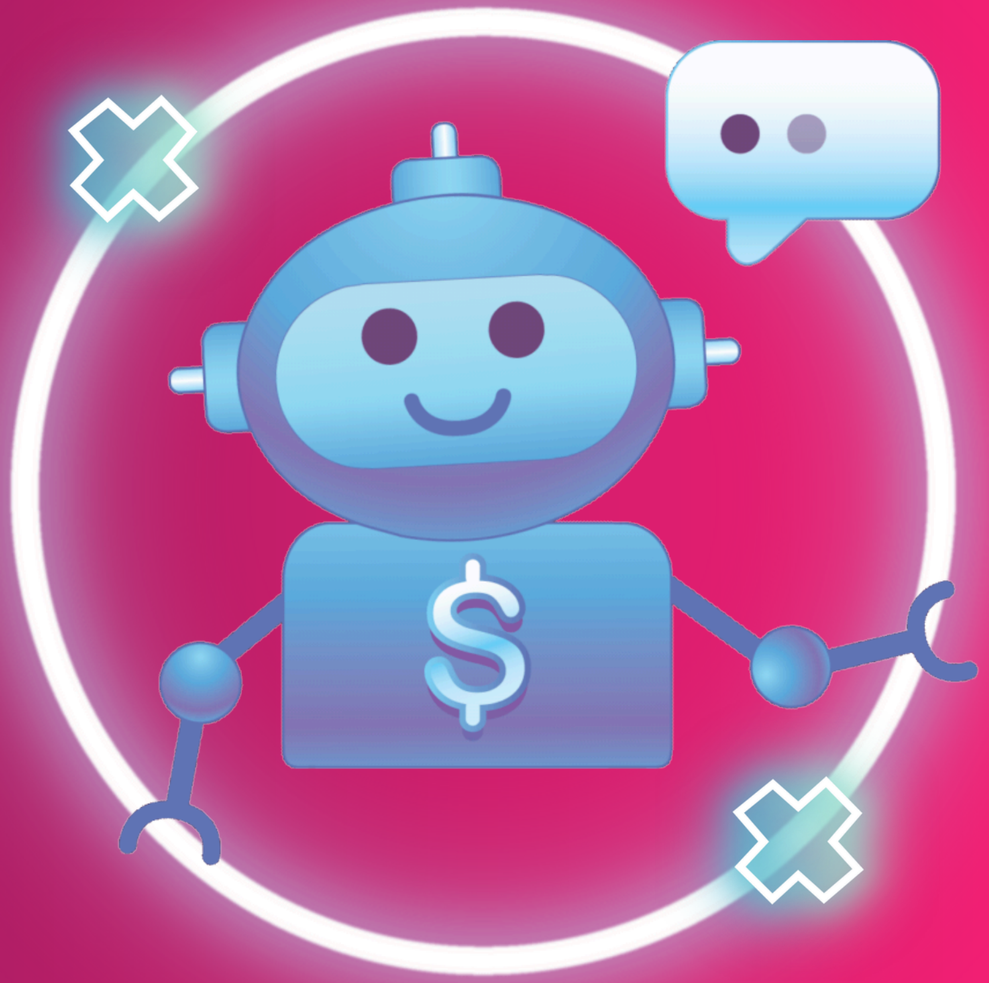
Query Categories

Category	**Example query**
Paraphrase	“I want to study how computers process human language”
Keyword	“machine learning”
Vague	“something about horror movies”
Prerequisite	“I just finished CS2040S what should I take next”

- **Paraphrase: Same intent, different wording. Tests whether retrieval generalizes past exact phrasing**
- **Keyword: Short, title-like queries. Tests literal/lexical matching strength.**
- **Vague: Underspecified, ambiguous. The hardest case, tests how retrieval handles little signal**
- **Prerequisite: Sequencing-style questions. Tests whether retrieval understands “next step” relationships, not just topic similarity**

YOUR TURN

try to “break” the engine. ask anything that you want to ask, even if it’s not related to NUSMods at all



THANK YOU!



ATTENDANCE



FEEDBACK FORM



Follow our socials to stay in the loop!

SDS Instagram

NUS.SDS

SDS Channel

@NUSSDS

SDS Workshop AY26/27 Github

SDS Workshop AY25/26 Github